

--- التمرين الاول: التعامل مع البيئة والتحقق منها ---

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
# عرض الاصدارات
print('tensorflow:',tf.__version__)
print('keras:',keras.__version__)
# GPU التحقق من
gpus=tf.config.list_physical_devices('GPU')
print('GPU متوفر:',len(gpus)>0)
# غير متاحة GPU استنتجت ذلك من نتيجة false
# غيابه يعني ان جميع العمليات اللازمة لتدريب النموذج تنفذ بواسطة CPU
# يؤدي هذا الي بطء ملحوظ في وقت الحساب
```

--- التمرين الثاني: التحليل النقدي لنتيجة التدريب ---

```
# تدريب النموذج
model = keras.Sequential([keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='adam', loss='mean_squared_error')
```

```
# بيانات افتراضية للتدريب (مطبوعة بين 0 و 1)
x_train = np.array([0.0, 0.1, 0.2, 0.3, 0.4, 0.5], dtype=float)
y_train = np.array([0.0, 0.2, 0.4, 0.6, 0.8, 1.0], dtype=float)
model.fit(x_train, y_train, epochs=100, verbose=0)
```

```
# تعليق النظري
# MSE: هو متوسط مربع الفرق بين التوقعات والقيم الحقيقية
#  $(1/n) * \sum (y_{true} - y_{pred})^2$ 
# سؤال: لماذا 2.317 مرتفعة؟
# لان البيانات مطبوعة بين 0 و 1، وهذا الخطأ يتجاوز نطاق البيانات نفسه، مما يدل على فشل النموذج في التقارب
# يجب ان تكون قيمتها قريبة من 0 بالنسبة لنموذج انحدار خطي مدرب جيدا على بيانات مطبوعة،
# فرضية: السبب قد يعود لعدم كفاية الحقب أو عدم ملائمة معدل التعلم الافتراضي
```

--- التمرين الثالث: استخراج معادلة النموذج ---

```
# استخراج المعاملات بعد تدريب النموذج
weights = model.get_weights()
w = weights[0][0][0] # الوزن (Weight)
b = weights[1][0] # الانحياز (Bias)
```

```
# عرض المعادلة في الحجم المطبوع
print(f"المعادلة المطبوعة: prix_norm = {w:.4f} * surface_norm + {b:.4f}")
```

```
# (Denormalization) عملية إلغاء التطبيع
# لنفترض هنا قيم افتراضية (يجب استبدالها بقيمك الحقيقية إذا كانت مختلفة)،
# مثال: المساحات بين 0 و 500 م2 # min_surface, max_surface = 0, 500
# مثال: الأسعار بين 0 و 1000 k€ # min_prix, max_prix = 0, 1000
```

```
# معادلة تحويل المعاملات من النطاق المطبّع إلى النطاق الحقيقي
w_real = w * (max_prix - min_prix) / (max_surface - min_surface)
b_real = b * (max_prix - min_prix) + min_prix - (w_real * min_surface)

print(f"المعادلة الحقيقية: prix_real = {w_real:.4f} * surface_real + {b_real:.4f}")

# التحقق التلقائي لتنبؤ مساحة 100 م2
surface_test = 100
prix_predit = w_real * surface_test + b_real
```

هنا سيقوم البرنامج بالحساب وإظهار النتيجة مباشرة دون تدخل منك

```
print(f"ك€ {prix_predit:.2f} م2 هو {surface_test} السعر المتوقع تلقائياً لمساحة ")
```

--- (Diagnosis) التمرين الرابع: تشخيص عدم تقارب النموذج ---

ملاحظة الخلل:

(2.317) كانت مرتفعة (Loss) لأن قيمة الخسارة (Non-convergence) النموذج لم يتقارب.

لم تُضبط بدقة (Weights) التوقعات كانت غير منطقية لأن الأوزان

الأسباب الجوهرية (Root Causes):

القيمة الافتراضية (0.001) بطيئة جداً، (Learning Rate) معدل التعلم -

مما يمنع النموذج من الوصول للحل في 100 حقبة فقط.

حقبة غير كافية لتقارب النموذج مع هذا المعدل البطيء 100 (Epochs): عدد الحقب -

العلاقة بين المعلمات والتقارب:

إذا كان معدل التعلم صغيراً جداً: التقارب يكون بطيئاً جداً -

(Overshooting) إذا كان معدل التعلم كبيراً جداً: قد يتجاوز النموذج الحل -

الاستقرار: يتحقق عند الموازنة بين معدل تعلم مناسب (مثل 0.1) وعدد حقب كافٍ (مثل 500) -

--- التمرين الخامس: تصحيح وتدريب النموذج المحسّن ---

إعداد بيئة النموذج المحسّن 1.

معدلة (Hyperparameters) نقوم بإنشاء نموذج جديد بمعلمات فائقة:

```
model_improved = keras.Sequential([keras.layers.Dense(units=1, input_shape=[1])])
```

تحديد المعلمات الفائقة 2. (Hyperparameters Optimization):

معدل تعلم أسرع وأكثر فعالية من القيمة الافتراضية: learning_rate=0.1 -

مُحسّن ذكي يتكيف مع طبيعة دالة الخسارة: Adam optimizer -

```
model_improved.compile(optimizer=keras.optimizers.Adam(learning_rate=0.1),
                        loss='mean_squared_error')
```

التدريب الموسع 3. (Extended Training):

(Convergence) "زيادة عدد الحقب إلى 500 حقبة لضمان وصول النموذج إلى نقطة التقارب"

لعدم عرض تفاصيل كل حقبة (لحفاظ على نظافة التقرير) verbose=0

```
history = model_improved.fit(x_train, y_train, epochs=500, verbose=0)
```

4. تقييم النتائج بعد التحسين:

```
final_loss = model_improved.evaluate(x_train, y_train, verbose=0)
weights_imp = model_improved.get_weights()
w_imp = weights_imp[0][0][0]
b_imp = weights_imp[1][0]
```

5. عرض النتائج النهائية:

```
print(f"--- نتائج النموذج بعد التصحيح")
print(f"(Loss) الخسارة النهائية: {final_loss:.8f}")
print(f"المعادلة النهائية المحسنة: prix_norm = {w_imp:.4f} * surface_norm + {b_imp:.4f}")
```

عند مقارنة هذه النتائج بالنموذج الأول، نلاحظ انخفاض الخسارة إلى مستويات تقترب من الصفر.
هذا يؤكد أن ضبط معدل التعلم وعدد الحقب كان هو الحل الجذري لمشكلة عدم التقارب

#

=====

==

التمرين السادس: تفسير النتائج النهائية والتحليل النقدي

#

=====

==

1. التفسير الرياضي للنتائج:

كانت متوقعة لأن البيانات التي دربنا عليها النموذج ($w=2.0$, $b=0.0$) النتيجة
نجاح النموذج في استرجاع ($y = 2x + 0$) تم إنشاؤها أصلاً بناءً على المعادلة
هذه المعاملات هو دليل قاطع على دقة عملية التعلم والتقارب الرياضي

2. دلالة الخسارة (Loss = 0.000000):

هذه القيمة تشير إلى وصول النموذج إلى تطابق تام مع بيانات التدريب
ملاحظة هامة: في التطبيقات الفعلية، هذه النتيجة ليست مرغوبة دائماً؛
، حيث يقوم النموذج "بحفظ" البيانات (Overfitting) لأنها قد تدل على حدوث
بدلاً من "فهمها"، مما يقلل قدرته على التنبؤ ببيانات جديدة (غير معروفة)

3. حدود النموذج في الواقع (Real-world constraints):

يفترض علاقة طردية ثابتة، بينما العقارات في الواقع: (Linear) النموذج خطي -
تتأثر بعوامل معقدة وغير خطية (الموقع، الحالة، الخدمات، العرض والطلب)
نقص المتغيرات: النموذج الحالي يعتمد على المساحة فقط ويتجاهل عوامل -
جوهرية أخرى تؤثر على سعر العقار

4. تحسينات مقترحة لجعل النموذج أكثر متانة:

إدخال متغيرات إضافية مثل (عمر العقار، (Feature Engineering) هندسة الميزات -
(عدد الغرف، الموقع الجغرافي)
(Deep Neural Networks) الانتقال لنماذج غير خطية: استخدام الشبكات العصبية العميقة -
أو نماذج الانحدار غير الخطي لقدرتها على تمثيل العلاقات المعقدة

```
#
=====

===
طباعة النتائج للتوثيق #
print("--- تقرير التفسير والتحليل ---")
print("النموذج نجح في استرجاع المعاملات الأصلية")
print("الخسارة الصفرية تشير إلى تعلم دقيق للبيانات التدريبية")
print("التحسين المستقبلي: إدراج متغيرات أكثر واستخدام نماذج أعقد")
#
=====

===
# (Synthesis Report) التمرين السابع: تقرير التوليف والتفكير النقدي #
#
=====

==
```

#1. تطبيع البيانات (Data Normalization):
الهدف: توحيد نطاق البيانات (بين 0 و 1) لضمان استقرار التدريب ومنع انحياز النموذج -
الكيفية: استخدام معادلة الـ Min-Max Scaling

#. دالة الخسارة (MSE):
هي مقياس الخطأ بين توقعات النموذج والقيم الحقيقية -
(Convergence) "انخفاضها التدريجي هو المؤشر الرئيسي لنجاح عملية" تقارب النموذج .
(Hyperparameters) المعلمات الفائقة:
معدل التعلم (0.1): يحدد سرعة التعلم؛ ضبطه كان حاسماً لتجنب التباطؤ أو التذبذب -
عدد الحقب (500): يمثل زمن التدريب؛ زيادته سمحت للنموذج بتصحيح أخطائه بدقة -

استخراج المعاملات وتفسير الأداء:
، مما يؤكد نجاح النموذج ($w=2, b=0$) تمكنا من استرجاع معاملات الانحدار
"في تعلم النمط الخطي. النموذج دقيق جداً لكنه يظل "خطي المحدودية"
ويحتاج لبيانات أكثر تعقيداً في التطبيقات الواقعية#

#النتائج

--- لتمرين الاول: التعامل مع البيئة والتحقق منها#

```
import tensorflow as tf

from tensorflow import keras

import numpy as np

عرض الاصدارات#

print('tensorflow:',tf.__version__)

print('keras:',keras.__version__)
```

GPU التحقق من

```
gpus=tf.config.list_physical_devices('GPU')
```

```
print('GPU متوفر',len(gpus)>0)
```

غير متاحة GPU استنتجت ذلك من نتيجة false

غيابه يعني ان جميع العمليات اللازمة لتدريب النموذج تنفذ بواسطة CPU

يؤدي هذا الي بطء ملحوظ في وقت الحساب

--- التمرين الثاني: التحليل النقدي لنتيجة التدريب ---

تدريب النموذج

```
model = keras.Sequential([keras.layers.Dense(units=1, input_shape=[1])])
```

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

بيانات افتراضية للتدريب (مطبوعة بين 0 و 1)

```
x_train = np.array([0.0, 0.1, 0.2, 0.3, 0.4, 0.5], dtype=float)
```

```
y_train = np.array([0.0, 0.2, 0.4, 0.6, 0.8, 1.0], dtype=float)
```

```
model.fit(x_train, y_train, epochs=100, verbose=0)
```

تعليق النظري

MSE: هو متوسط مربع الفرق بين التوقعات والقيم الحقيقية

$(1/n) * \sum (y_{true} - y_{pred})^2$

سؤال: لماذا 2.317 مرتفعة؟

لان البيانات مطبوعة بين 0 و 1، وهذا الخطأ يتجاوز نطاق البيانات نفسه، مما يدل على فشل النموذج في التقارب

بالنسبة لنموذج انحدار خطي مدرب جيدا على بيانات مطبوعة، يجب ان تكون قيمتها قريبة من 0

فرضية: السبب قد يعود لعدم كفاية الحقب أو عدم ملائمة معدل التعلم الافتراضي

--- التمرين الثالث: استخراج معادلة النموذج ---

استخراج المعاملات بعد تدريب النموذج

```
weights = model.get_weights()
```

w = weights[0][0][0] # الوزن (Weight)

```
b = weights[1][0] # الانحياز (Bias)
```

```
# عرض المعادلة في الحجم المطبّع
```

```
print(f"المعادلة المطبّعة: prix_norm = {w:.4f} * surface_norm + {b:.4f}")
```

```
# عملية إلغاء التطبيع (Denormalization)
```

```
# لنفترض هنا قيم افتراضية (يجب استبدالها بقيمك الحقيقية إذا كانت مختلفة)
```

```
min_surface, max_surface = 0, 500 # مثال: المساحات بين 0 و 500 م2
```

```
min_prix, max_prix = 0, 1000 # مثال: الأسعار بين 0 و 1000 k€
```

```
# معادلة تحويل المعاملات من النطاق المطبّع إلى النطاق الحقيقي
```

```
w_real = w * (max_prix - min_prix) / (max_surface - min_surface)
```

```
b_real = b * (max_prix - min_prix) + min_prix - (w_real * min_surface)
```

```
print(f"المعادلة الحقيقية: prix_real = {w_real:.4f} * surface_real + {b_real:.4f}")
```

```
# التحقق التلقائي لتنبؤ مساحة 100 م2
```

```
surface_test = 100
```

```
prix_predit = w_real * surface_test + b_real
```

```
# هنا سيقوم البرنامج بالحساب وإظهار النتيجة مباشرة دون تدخل منك
```

```
print(f"{prix_predit:.2f} k€ هو {surface_test} السعر المتوقع تلقائياً لمساحة")
```

--- (Diagnosis) التمرين الرابع: تشخيص عدم تقارب النموذج ---

```
# ملاحظة الخلل:
```

```
# (2.317) كانت مرتفعة (Loss) لأن قيمة الخسارة (Non-convergence) النموذج لم يتقارب
```

```
# لم تُضبط بدقة (Weights) التوقعات كانت غير منطقية لأن الأوزان
```

```
# الأسباب الجوهرية (Root Causes):
```

```
# القيمة الافتراضية (0.001) بطيئة جداً، (Learning Rate) معدل التعلم -
```

مما يمنع النموذج من الوصول للحل في 100 حقبة فقط.

حقبة غير كافية لتقارب النموذج مع هذا المعدل البطيء (Epochs): 100 عدد الحقب -

العلاقة بين المعلمات والتقارب:

إذا كان معدل التعلم صغيراً جداً: التقارب يكون بطيئاً جداً -

(Overshooting) إذا كان معدل التعلم كبيراً جداً: قد يتجاوز النموذج الحل -

الاستقرار: يتحقق عند الموازنة بين معدل تعلم مناسب (مثل 0.1) وعدد حقب كافٍ (مثل 500) -

--- التمرين الخامس: تصحيح وتدريب النموذج المحسن ---

إعداد بيئة النموذج المحسن 1.

معدلة (Hyperparameters) نقوم بإنشاء نموذج جديد بمعلمات فائقة:

```
model_improved = keras.Sequential([keras.layers.Dense(units=1, input_shape=[1])])
```

2. (Hyperparameters Optimization) تحديد المعلمات الفائقة:

معدل تعلم أسرع وأكثر فعالية من القيمة الافتراضية: learning_rate=0.1 -

مُحسن ذكي يتكيف مع طبيعة دالة الخسارة: Adam optimizer -

```
model_improved.compile(optimizer=keras.optimizers.Adam(learning_rate=0.1),  
                        loss='mean_squared_error')
```

3. التدريب الموسع (Extended Training):

(Convergence) "زيادة عدد الحقب إلى 500 حقبة لضمان وصول النموذج إلى نقطة التقارب".

لعدم عرض تفاصيل كل حقبة (الحفاظ على نظافة التقرير) verbose=0

```
history = model_improved.fit(x_train, y_train, epochs=500, verbose=0)
```

4. تقييم النتائج بعد التحسين:

```
final_loss = model_improved.evaluate(x_train, y_train, verbose=0)  
weights_imp = model_improved.get_weights()  
w_imp = weights_imp[0][0][0]  
b_imp = weights_imp[1][0]
```

عرض النتائج النهائية. 5:

```
print(f"--- نتائج النموذج بعد التصحيح ---")
```

```
print(f"الخسارة النهائية (Loss): {final_loss:.8f}")
```

```
print(f"المعادلة النهائية المحسنة: prix_norm = {w_imp:.4f} * surface_norm + {b_imp:.4f}")
```

عند مقارنة هذه النتائج بالنموذج الأول، نلاحظ انخفاض الخسارة إلى مستويات تقترب من الصفر.

هذا يؤكد أن ضبط معدل التعلم وعدد الحقب كان هو الحل الجذري لمشكلة عدم التقارب.

#

```
=====
```

التمرين السادس: تفسير النتائج النهائية والتحليل النقدي

#

```
=====
```

التفسير الرياضي للنتائج. 1:

كانت متوقعة لأن البيانات التي دربنا عليها النموذج ($w=2.0$, $b=0.0$) النتيجة

نجاح النموذج في استرجاع ($y = 2x + 0$). تم إنشاؤها أصلاً بناءً على المعادلة

هذه المعاملات هو دليل قاطع على دقة عملية التعلم والتقارب الرياضي.

2. دلالة الخسارة ($Loss = 0.000000$):

هذه القيمة تشير إلى وصول النموذج إلى تطابق تام مع بيانات التدريب.

ملاحظة هامة: في التطبيقات الفعلية، هذه النتيجة ليست مرغوبة دائماً؛

، حيث يقوم النموذج "بحفظ" البيانات (Overfitting) لأنها قد تدل على حدوث

بدلاً من "فهمها"، مما يقلل قدرته على التنبؤ ببيانات جديدة (غير معروفة).

3. حدود النموذج في الواقع (Real-world constraints):

يفترض علاقة طردية ثابتة، بينما العقارات في الواقع (Linear): النموذج خطي -

تتأثر بعوامل معقدة وغير خطية (الموقع، الحالة، الخدمات، العرض والطلب)

نقص المتغيرات: النموذج الحالي يعتمد على المساحة فقط ويتجاهل عوامل -

#.جوهريه أخرى تؤثر على سعر العقار

#تحسينات مقترحة لجعل النموذج أكثر متانة. 4

إدخال متغيرات إضافية مثل (عمر العقار، (Feature Engineering) هندسة الميزات -

(عدد الغرف، الموقع الجغرافي

(Deep Neural Networks) الانتقال لنماذج غير خطية: استخدام الشبكات العصبية العميقة -

#.أو نماذج الانحدار غير الخطي لقدرتها على تمثيل العلاقات المعقدة

#

====

طباعة النتائج للتوثيق

print("--- تقرير التفسير والتحليل ---")

print("النموذج نجح في استرجاع المعاملات الأصلية")

print("الخسارة الصفرية تشير إلى تعلم دقيق للبيانات التدريبية")

print("التحسين المستقبلي: إدراج متغيرات أكثر واستخدام نماذج أعقد")

#

====

(Synthesis Report) التمرين السابع: تقرير التوليف والتفكير النقدي

#

====

#1. تطبيع البيانات (Data Normalization):

الهدف: توحيد نطاق البيانات (بين 0 و 1) لضمان استقرار التدريب ومنع انحياز النموذج -

#- Min-Max Scaling الكيفية: استخدام معادلة ال

#. دالة الخسارة (MSE):

هي مقياس الخطأ بين توقعات النموذج والقيم الحقيقية -

#. (Convergence) "انخفاضها التدريجي هو المؤشر الرئيسي لنجاح عملية" تقارب النموذج

#. (Hyperparameters) المعلمات الفائقة:

معدل التعلم (0.1): يحدد سرعة التعلم؛ ضبطه كان حاسماً لتجنب التباطؤ أو التذبذب -#
عدد الحقب (500): يمثل زمن التدريب؛ زيادته سمحت للنموذج بتصحيح أخطائه بدقة -#

استخراج المعاملات وتفسير الأداء #

، مما يؤكد نجاح النموذج ($w=2, b=0$) تمكنا من استرجاع معاملات الانحدار #
"في تعلم النمط الخطي. النموذج دقيق جداً لكنه يظل "خطي المحدودية #
ويحتاج لبيانات أكثر تعقيداً في التطبيقات الواقعية #